Python Questions For Freshers

Q1. What is Python? What are the benefits of using Python?

✅ What is Python?

Python is a **high-level, interpreted, and general-purpose programming language** known for its **simple syntax and readability**. It supports multiple programming paradigms like **object-oriented, procedural, and functional programming**, making it versatile and easy to use for beginners as well as professionals.

✅ Benefits of Using Python

Some key benefits of Python are:

- **Easy to learn and write**
  Python's clean and simple syntax makes development faster and reduces learning time.
- **Large standard library & strong community support**
  It has extensive built-in modules and a huge community that provides libraries, frameworks, and help.
- **Cross-platform language**
  Python works on Windows, macOS, Linux, and can run on different systems without major changes.
- **Rich ecosystem of libraries and frameworks**
  Popular in data science, machine learning, web development, automation, etc. (e.g., NumPy, Pandas, Django, Flask, TensorFlow)
- **Rapid development**
  Faster prototyping and development compared to many other languages.
- **Highly versatile**
  Used in multiple domains like:
    - Web development
    - Data analysis
    - Machine learning & AI
    - Automation/Scripting
    - Cybersecurity
    - Software development

✅ One-Line Interview Closing Statement

"Overall, Python is popular because it combines simplicity, flexibility, and powerful libraries, which makes development fast and efficient."

2. What is a dynamically typed language?

A dynamically typed language is a language where the data type of a variable is determined at runtime (while the program is running), not at compile time.

In simple words:

👉 **You don't need to declare the type of a variable before using it**
👉 **The interpreter automatically decides the type based on the value assigned**

Example in Python:

```
x = 10      # x is an integer
x = "Hello" # now x becomes a string
```

✅ **Key Points for Interview**

- **Type checking happens at runtime**
- **No need to specify variable types explicitly**
- **Variables can change their type during execution**

✅ **Advantages**

- **Faster development**
- **Less code and easier to write**
- **More flexible**

✅ **Disadvantages**

- **Errors related to types may appear at runtime**
- **Slightly slower performance compared to statically typed languages**

✅ **One-line closing**

"Python is dynamically typed, which makes coding faster and easier, but requires careful handling of variable types."

Q3. What is an Interpreted language?

An interpreted language is a programming language where the code is executed line by line by an interpreter at runtime, instead of being converted into machine code beforehand.

In simple words:

👉 **The code runs directly without compiling**
👉 **The interpreter reads, translates, and executes the program step by step**

✅ **Example**

Python, JavaScript, Ruby, PHP are common interpreted languages.

✅ **How it works (simple explanation)**

1. **You write the code**
2. **The interpreter reads one line**
3. **Converts it to machine instructions**
4. **Executes it immediately**

✅ **Advantages**

- **Easy to debug (errors appear where they happen)**
- **Faster development and testing**
- **Platform independent (can run on different OS)**

✅ **Disadvantages**

- **Slower execution compared to compiled languages**
- **Errors may appear at runtime**

✅ **One-line closing**

"Python is an interpreted language, which allows quick development and easy debugging, but may have slower performance compared to compiled languages like C or C++."

Q4. What is PEP 8 and why is it important?

PEP 8 is the official style guide for Python code.
It stands for Python Enhancement Proposal 8, and it provides rules and best practices for writing clean, readable, and consistent Python code.

It covers things like:

- **Naming conventions (variables, functions, classes)**
- **Indentation (4 spaces)**
- **Line length (max 79 characters)**
- **Spacing and blank lines**

- **Imports formatting**
- **Code layout and structure**

✅ **Why is PEP 8 important?**

PEP 8 is important because it:

- **Improves code readability**
- **Makes code consistent across projects and teams**
- **Makes collaboration easier in teams and open-source projects**
- **Reduces bugs caused by messy or unclear code**
- **Helps maintain code in the long term**

✅ **One-line closing**

"PEP 8 ensures that Python code looks clean and consistent, making it easier to read, understand, and maintain."

Q5. What is Scope in Python?

✅ **What is Scope in Python?**

Scope in Python refers to the region of the program where a variable is accessible.
It decides which variables can be used where during program execution.

Python follows the LEGB rule to find variables:

1. **Local Scope – variables inside a function**
2. **Enclosing Scope – variables in outer functions (nested functions)**
3. **Global Scope – variables defined at the top level of the script**
4. **Built-in Scope – predefined names like print, len, range**

✅ **Why is Scope important?**

Scope is important because it:

- **prevents variable name conflicts**
- **controls variable accessibility**
- **improves memory management**
- **makes code easier to maintain and debug**

✅ **Bonus (Professional touch)**

Python also provides:

- **global keyword → to modify global variables inside a function**
- **nonlocal keyword → to modify enclosing variables in nested functions**

✅ **One-line closing**

"Scope defines where a variable can be accessed in Python, and Python resolves variables using the LEGB rule."

Q6. What are lists and tuples? What is the key difference between the two?

✅ **What are Lists in Python?**

A list is a mutable (changeable) collection in Python used to store multiple items in a single variable.
You can:

- **add elements**
- **remove elements**
- **modify elements**

Example:

my_list = [1, 2, 3]

✅ **What are Tuples in Python?**

A tuple is an immutable (unchangeable) collection in Python.
Once a tuple is created, you cannot add, remove, or modify its elements.

Example:

my_tuple = (1, 2, 3)

✅ **Key Difference**

The main difference is:

👉 **Lists are mutable**
👉 **Tuples are immutable**

✅ **Additional Professional Points**

- **Lists are slower than tuples because they allow modifications**
- **Tuples are faster and more memory efficient**
- **Tuples are often used for fixed data (like coordinates, dates, configuration)**

✅ **One-line closing**

"Lists can be changed after creation, while tuples cannot, which makes tuples faster and more memory efficient."

Q7. What are the common built-in data types in Python?

✅ Common Built-in Data Types in Python

Python provides several built-in data types, mainly:

✅ 1. Numeric Types

- int → integers (10, -5)
- float → decimal numbers (3.14)
- complex → complex numbers (2+3j)

✅ 2. Sequence Types

- str → strings ("Hello")
- list → mutable collection [1, 2, 3]
- tuple → immutable collection (1, 2, 3)
- range → sequence of numbers (range(5))

✅ 3. Mapping Type

- dict → key-value pairs { "name": "John" }

✅ 4. Set Types

- set → unordered unique elements {1, 2, 3}
- frozenset → immutable set

✅ 5. Boolean Type

- bool → True / False

✅ 6. None Type

- None → represents no value / null

✅ One-line closing

"Python has rich built-in data types such as numeric, sequence, mapping, set, boolean, and NoneType, which help in storing and managing different kinds of data efficiently."

Q8. What is pass in Python?

✅ **What is pass in Python?**

pass is a placeholder statement in Python that does nothing when executed.
It is used when a statement is syntactically required, but you don't want to write any code yet.

In simple words:

👉 **Python needs a line of code there**
👉 **You don't want to execute anything**
➡️ **So you use pass**

✅ **Why is pass important?**

Without pass, Python will give an error if a block is empty.

✅ **One-line closing**

"pass is a placeholder statement in Python that allows you to write empty code blocks without causing errors."

Q9. What are modules and packages in Python?

✅ **What is a Module in Python?**

A module is a single Python file (.py) that contains code such as:

- **functions**
- **classes**
- **variables**
- **executable code**

It helps in organizing and reusing code.

✅ **What is a Package in Python?**

A package is a collection of modules organized in a folder.
It contains:

- **multiple .py files (modules)**
- **an __init__.py file (Python treats the folder as a package)**

✅ **Key Difference**

👉 Module = single file (.py)
👉 Package = folder containing multiple modules

✅ Why are they important?

They help in:

- code reusability
- better organization
- avoiding name conflicts
- easier project structure
- modular programming

✅ One-line closing

"A module is a single Python file, while a package is a collection of modules that helps organize large Python projects."

Q10. What are global, protected and private attributes in Python?

✅ Global Attributes

Global attributes (or global variables) are:

- defined outside any class or function
- accessible throughout the program

✅ Protected Attributes

Protected attributes are intended for internal use within a class and its subclasses.

In Python, they are defined using a single underscore _ prefix:

```
class A:
    _value = 20   # protected attribute
```

✅ Accessible inside the class
✅ Accessible in subclasses
❌ Not recommended to access from outside (but still possible)

👉 Python does NOT enforce protection strictly, it's based on convention.

✅ Private Attributes

Private attributes are meant to be fully hidden within a class.

Defined using a double underscore __ prefix:

class A:

__secret = 30 # private attribute

✅ **Accessible only inside the class**
❌ **Not directly accessible outside**

Python applies name mangling:

__secret becomes _A__secret internally

✅ **One-line closing**

"In Python, global attributes are accessible everywhere, protected attributes are intended for use within a class and its subclasses, and private attributes use name mangling to restrict access inside the class."

Q11. What is the use of self in Python?

✅ **What is self in Python?**

self is a reference to the current object (instance) of a class.
It is used to:

- **access instance variables**
- **access instance methods**
- **differentiate between class attributes and local variables**

In simple words:

👉 **self represents the object on which a method is being called.**

✅ **Why do we need self?**

Inside a class, Python needs a way to know:

- **which object's data to use**
- **which object's attribute to modify**

self provides that connection.

✅ **Example**

```
class Student:
    def __init__(self, name):
        self.name = name   # assigning to instance
    def show(self):
        print(self.name)
```

obj = Student("Yogi")

obj.show()

Here:

- **self.name belongs to the object obj**
- **without self, Python wouldn't know which object's name to use**

✅ **Important Interview Point**

self is not a keyword in Python.

It's just a naming convention.
You can use another name, but using self is standard and recommended.

✅ **One-line closing**

"self refers to the current object in a class and is used to access instance variables and methods."

Q12. What is __init__?

✅ **What is __init__ in Python?**

__init__ is a special method (constructor) in Python that automatically runs when an object of a class is created.

Its main purpose is to:

- **initialize object attributes**
- **set initial values**
- **prepare the object for use**

✅ **Example**

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
obj = Student("Yogi", 22)
```

Here, __init__ assigns values to the object when it is created.

✅ **Why is __init__ important?**

It helps to:

- **initialize data for each object**
- **avoid manually setting attributes later**
- **make object creation cleaner and organized**

✅ **Important Interview Points**

- **__init__ is automatically called**
- **It is similar to constructors in other OOP languages**
- **It must have self as the first parameter**
- **It can take additional parameters**

✅ **One-line closing**

"__init__ is a constructor method in Python that initializes object attributes automatically when an object is created."

Q13. What is break, continue and pass in Python?

break is used to stop the loop immediately and exit from it.

continue skips the current iteration and moves to the next iteration of the loop.

pass is a placeholder statement that does nothing.
Used when a statement is required syntactically but you don’t want to write any code yet.

15. What is docstring in Python?

- Documentation string or docstring is a multiline string used to document a specific code segment.
- The docstring should describe what the function or method does.

16. What is slicing in Python?

- As the name suggests, 'slicing' is taking parts of.
- Syntax for slicing is **[start : stop : step]**
- **start** is the starting index from where to slice a list or tuple
- **stop** is the ending index or where to sop.
- **step** is the number of steps to jump.
- Default value for **start** is 0, **stop** is number of items, **step** is 1.
- Slicing can be done on **strings, arrays, lists**, and **tuples**.

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
print(numbers[1 : : 2])  #output : [2, 4, 6, 8, 10]
```

17. Explain how can you make a Python Script executable on Unix?

- Script file must begin with **#!/usr/bin/env python**

Q18. What is the difference between Python Arrays and lists?

✅ **Difference Between Python Arrays and Lists**

✅ **Lists**

- **Can store different data types in a single list (example: int, float, string together)**
- **Dynamic size (can grow or shrink)**
- **More flexible and widely used**
- **Slightly slower and less memory efficient**

✅ **Arrays**

In Python, arrays (from the array module):

- **Store only one data type**
- **More memory efficient**
- **Faster for numerical operations**

✅ **Key Interview Difference**

👉 **Lists are more flexible and can store mixed data types**
👉 **Arrays store only one data type and are more memory efficient and faster for numerical data**

✅ **Important Professional Point**

In Data Science, when people say "array", they often mean NumPy arrays, which are:

- **much faster than lists**
- **support vectorized operations**
- **used for mathematical and scientific computing**

✅ **One-line closing**

"Lists are flexible and can store mixed data, while arrays store only one data type and are more memory-efficient and faster for numerical operations."

Q19. What is exception handling in python?

✅ **What is Exception Handling in Python?**

Exception handling in Python is a mechanism that allows you to handle runtime errors gracefully, so the program does not crash.
It lets you detect errors, handle them, and continue execution.

In simple words:

👉 **When an error occurs, instead of stopping the program, Python lets you manage it safely.**

✅ **Why is it important?**

Exception handling helps to:

- **prevent program crashes**
- **provide meaningful error messages**
- **maintain smooth program flow**
- **handle unexpected situations**

✅ **How exception handling works**

Python uses:

- **try → code that may cause an error**
- **except → handles the error**
- **else → runs if no error occurs**
- **finally → always runs (cleanup code)**

✅ **Example**

```
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
finally:
    print("Done")
```

✅ **Important Interview Points**

- **Exceptions are runtime errors**
- **Prevent program termination**
- **raise keyword is used to manually raise an exception**

✅ **One-line closing**

"Exception handling in Python allows programs to handle runtime errors gracefully using try-except blocks, preventing crashes and improving reliability.

Python interview questions (Advanced)

Q19 . How is memory managed in Python?

Memory management refers to process of allocating and deallocating memory to a program while it runs. Python handles memory management automatically using mechanisms like reference counting and garbage collection, which means programmers do not have to manually manage memory.

Let's explore how Python automatically manages memory using garbage collection and reference counting.

Garbage Collection

It is a process in which Python automatically frees memory occupied by objects that are no longer in use.

- If an object has no references pointing to it (i.e., nothing is using it), garbage collector removes it from memory.
- This ensures that unused memory can be reused for new objects.

For more information, refer to Garbage Collection in Python

Reference Counting

It is one of the primary memory management techniques used in Python, where:

- Every object keeps a reference counter, which tells how many variables (or references) are currently pointing to that object.
- When a new reference to the object is created, counter increases.
- When a reference is deleted or goes out of scope, counter decreases.
- If the counter reaches zero, it means no variable is using the object anymore, so Python automatically deallocates (frees) that memory.

Q20. What are Python namespaces? Why are they used?

In Python, a **namespace is a container that maps names to objects**. It stores the identifiers used in a program — such as variable names, function names, and class names — and keeps track of the objects they refer to in memory.

Namespaces are used to **avoid naming conflicts** and to **organize code so the same name can be used in different parts of a program without interference**. For example, a variable inside a function can have the same name as a global variable because they exist in different namespaces.

Python has **four main types of namespaces**:

- **Built-in namespace** – contains built-in functions like len, print.
- **Global namespace** – created for each module or script.
- **Local namespace** – created inside functions.
- **Enclosing namespace** – used for nested functions.

To resolve a name, Python follows the **LEGB rule**: Local → Enclosing → Global → Built-in.

Q21. What is Scope Resolution in Python?

Scope resolution in Python refers to the **order in which Python searches for a variable or name when it is referenced**. When a name is used, Python does not immediately know where it is defined, so it follows a fixed order to find it. This search order is known as the **LEGB rule**:

1. **L – Local scope** → Inside the current function/block
2. **E – Enclosing scope** → In the outer function (for nested functions)
3. **G – Global scope** → At the module level
4. **B – Built-in scope** → Predefined names like len, print

Scope resolution ensures that the correct variable is used and helps avoid conflicts between names defined in different parts of the program.

Q22. What are Decorators in Python?

Decorators in Python are essentially functions that add functionality to an existing function in Python without changing the structure of the function itself. They are represented the @decorator_name in Python and are called in a bottom-up fashion.

```python
# decorator function to capitalize names
def names_decorator(function):
    def wrapper(arg1, arg2):
        arg1 = arg1.capitalize()
        arg2 = arg2.capitalize()
        string_hello = function(arg1, arg2)
        return string_hello
    return wrapper
@names_decorator
def say_hello(name1, name2):
    return 'Hello ' + name1 + '! Hello ' + name2 + '!'
say_hello('sara', 'ansh')   # output => 'Hello Sara! Hello Ansh!'
```

The beauty of the decorators lies in the fact that besides adding functionality to the output of the method, they can even accept arguments for functions and can further modify those arguments before passing it to the function itself. The inner nested function, i.e. 'wrapper' function, plays a significant role here.

Q23. What are Dict and List comprehensions?

List and Dict comprehensions are **concise and elegant ways to create lists and dictionaries in a single line using a loop**, instead of writing multiple lines of code.

- **List Comprehension**

List comprehension provides a shorter syntax to construct a new list by applying an expression to each item in an iterable ex. squares = [x*x for x in range(5)]

- **Dictionary Comprehension**

Dictionary comprehension allows us to create dictionaries in a compact way by iterating over an iterable and defining key–value pairs.

Example: squares_dict = {x: x*x for x in range(5)}

```python
a = [1, 2, 3]
b = [7, 8, 9]
[(x + y) for (x,y) in zip(a,b)]  # parallel iterators
# output => [8, 10, 12]
[(x,y) for x in a for y in b]    # nested iterators
# output => [(1, 7), (1, 8), (1, 9), (2, 7), (2, 8), (2, 9), (3, 7), (3, 8), (3, 9)]
```

- **Flattening a multi-dimensional list**

Q24. What is lambda in Python? Why is it used?

A **lambda** in Python is an **anonymous (nameless) function** defined using the lambda keyword instead of def.
It can take any number of arguments but can contain only **one expression**.

Lambdas are mainly used when we need a **small, short-term function** without formally defining a full function.

Example:

square = lambda x: x * x

print(square(5)) # Output: 25

🔹 **Why lambda functions are used**

Lambdas are useful when:

- A function is required only temporarily
- Writing a full function using def would be unnecessary
- Used together with functions like map(), filter(), sorted(), reduce() etc.

Q25. How do you copy an object in Python?

In Python, objects can be copied in two ways:

1️⃣ **Shallow Copy**

Creates a new object but **does not create copies of nested objects**.
Both the original and the copy share references to nested objects.

Done using:

import copy

shallow_copy = copy.copy(obj)

2️⃣ **Deep Copy**

Creates a new object **and recursively copies all nested objects**.
Original and copy become completely independent.

Done using:

import copy

deep_copy = copy.deepcopy(obj)

Q26. What is the difference between xrange and range in Python?

xrange() and range() are quite similar in terms of functionality. They both generate a sequence of integers, with the only difference that range() returns a Python list, whereas, xrange() returns an xrange object. So how does that make a difference? It sure does, because unlike range(), xrange() doesn't generate a static list, it creates the value on the go. This technique is commonly used with an object-type generator and has been termed as "yielding". Yielding is crucial in applications where memory is a constraint. Creating a static list as in range() can lead to a Memory Error in such

conditions, while, xrange() can handle it optimally by using just enough memory for the generator (significantly less in comparison).

Q 27. What is pickling and unpickling?

Pickling is the process of **converting a Python object into a byte stream (serialization)** so it can be stored in a file or transferred over a network.

Unpickling is the **reverse process**, where the byte stream is **converted back into the original Python object (deserialization)**.

Python provides the pickle module to perform pickling and unpickling.

- **Example**

```
import pickle

data = {"name": "John", "age": 25}

# Pickling

with open("data.pkl", "wb") as file:

    pickle.dump(data, file)

# Unpickling

with open("data.pkl", "rb") as file:

    loaded_data = pickle.load(file)
```

- **Why pickling is used**

Pickling is mainly used for:

- Saving Python objects to disk
- Sending objects between systems
- Model storage in machine learning

Q 28. What are generators in Python?

Generators in Python are **special functions that return values one at a time using the yield keyword instead of return**.
They do **not store the entire sequence in memory**, but **generate values on the fly**, making them very memory-efficient for handling large datasets or infinite sequences.

A generator is an **iterator**, meaning it can be looped over, but it **produces values only when needed (lazy evaluation)**.

- **Example**

```
def my_generator():
    for i in range(5):
        yield i
for num in my_generator():
    print(num)
```

Output: 01234

- **Why generators are used**

Generators are preferred because:

- They **save memory** (don't store all values at once)
- They **improve performance** for large data processing
- They **support infinite sequences**
- They **allow lazy evaluation**

Q 29. What is PYTHONPATH in Python?

PYTHONPATH is an **environment variable** in Python that tells the Python interpreter **where to look for modules and packages** when importing them.

When you run an import statement, Python searches for the module in:

1. The current working directory
2. The directories listed in PYTHONPATH
3. The default installation directories

So, by setting PYTHONPATH, we can make Python recognize **custom module locations** that are not part of the standard library or site-packages.

- **Why PYTHONPATH is useful**

- To import modules/packages from custom directories
- To support large projects with multiple module locations
- To avoid installation of local modules in site-packages

Q 30. What is the use of help() and dir() functions?

help() and dir() are **built-in functions used for code inspection and debugging**.

- **help() function**

help() displays the **documentation of modules, classes, functions, and methods**.
It is used to understand what an object does and how to use it.

Example:

help(print)

This will show detailed information about the print() function.

- **dir() function**

dir() returns **a list of all valid attributes, methods, and variables** associated with an object, module, or class.

Example:

dir(list)

This will display all functions and attributes available for lists.

Q 31. What is the difference between .py and .pyc files?

.py files contain the source code of a program. Whereas, .pyc file contains the bytecode of your program. We get bytecode after compilation of .py file (source code). .pyc files are not created for all the files that you run. It is only created for the files that you import. Before executing a python program python interpreter checks for the compiled files. If the file is present, the virtual machine executes it. If not found, it checks for .py file. If found, compiles it to .pyc file and then python virtual machine executes it. Having .pyc file saves you the compilation time.

Q 32. How Python is interpreted?

Python is an **interpreted language**, meaning its code is executed line by line rather than being compiled directly into machine code.
However, Python internally follows a **two-step execution process**:

1. **Compilation to Bytecode**
   When a .py file is executed, Python first **compiles the source code into bytecode** — a low-level, platform-independent representation.
2. **Execution by the Python Virtual Machine (PVM)**
   The **Python Virtual Machine (PVM)** then **interprets the bytecode line by line** and executes it on the hardware.

So Python is interpreted, but it still performs a compilation step internally — it compiles to bytecode before interpretation.

- **Key points to mention in interview**
  - Python does **not** convert source code directly to machine code.
  - Instead, it converts it to **bytecode**, which is then **interpreted by PVM**.
  - Bytecode may be saved as .pyc files for faster future execution.

🧠 **One-line summary**

Python compiles source code into bytecode, and the Python Virtual Machine interprets this bytecode line by line to run the program.

Q 33. How are arguments passed by value or by reference in python?

Python uses **neither pure pass-by-value nor pure pass-by-reference**.
Instead, Python uses something called **pass-by-object-reference** (also known as **pass-by-assignment**).

When a function is called:

- **The reference (address) of the object is passed to the function**, not the actual object itself.
- But **whether the change affects the original object depends on the object's mutability**.

🔹 **Explanation with example**

```
def modify_list(lst):
    lst.append(100)
my_list = [1, 2, 3]
modify_list(my_list)
print(my_list)
```

Output:

[1, 2, 3, 100]

Because lists are **mutable**, changes inside the function affect the original object.

```
def modify_num(n):
        n = n + 1
        x = 10
        modify_num(x)
        print(x)
```

Output:

10

Because integers are **immutable**, the original variable does not change.

🧠 **Final takeaway (best one-liner for interview)**

Python passes arguments by **object reference**: if the object is mutable it can be modified inside the function, and if it is immutable it cannot.

34. What are iterators in Python?

An **iterator** in Python is an object that **allows sequential access to elements, one at a time**, without exposing the underlying data structure.
It follows the **iterator protocol**, which means it must implement:

- __iter__() → returns the iterator object itself
- __next__() → returns the next value and raises StopIteration when no items are left

---

🔹 **Example**

```
nums = [1, 2, 3]

it = iter(nums)


print(next(it))  # 1

print(next(it))  # 2

print(next(it))  # 3
```

🔹 **Why iterators are used**

- Memory-efficient traversal of large datasets
- Iterate over custom objects
- Power lazy evaluation (generate items on demand)

🧠 **One-line summary**

An iterator is an object that implements __iter__() and __next__(), allowing iteration over data one value at a time without loading everything into memory.

36. Explain split() and join() functions in Python?

split() and join() are commonly used string functions for breaking and combining strings.

split() **splits a string into a list of substrings** based on the specified separator (default is space).

join() **combines a list of strings into a single string** using a specified separator.

37. What does *args and **kwargs mean?

In Python, *args and **kwargs are used to pass a **variable number of arguments** to a function.

- ***args → Variable number of positional arguments**

*args allows a function to accept **any number of positional arguments** as a tuple.

Example:

```
def add(*args):
    return sum(args)
print(add(1, 2, 3))  # 6
```

- ****kwargs → Variable number of keyword arguments**

kwargs allows a function to accept **any number of keyword arguments as a dictionary.

Example:

```
def display(**kwargs):
    print(kwargs)
display(name="John", age=25)
```

Output:

```
{'name': 'John', 'age': 25}
```

38. What are negative indexes and why are they used?

Negative indexing in Python allows access to sequence elements **from the end instead of the beginning**.
While positive indexes start from 0, negative indexes start from -1 (last element), -2 (second last), and so on.

- **Why negative indexing is used**
  - To **access elements from the end quickly**
  - Useful when the **length of the sequence is unknown**
  - Eliminates need to calculate len(arr) – index

Python OOPS Interview Questions

39. How do you create a class in Python?

A class in Python is created using the **class keyword**.
A class is a blueprint for creating objects and defines **attributes (variables) and methods (functions)** that the objects will have.

🔹 **Example**

```
class Person:
    def __init__(self, name, age):   # constructor
        self.name = name
        self.age = age
    def display(self):               # method
        print(f"Name: {self.name}, Age: {self.age}")
# creating an object of the class
p1 = Person("John", 25)
p1.display()
```

🔹 **Explanation**

- class Person: → defines the class
- __init__() → constructor that initializes object attributes
- self → refers to the current object
- p1 = Person("John", 25) → creating an object
- p1.display() → calling a method of the class

40. How does inheritance work in python? Explain it with an example.

*Inheritance in Python is an Object-Oriented Programming (OOP) concept that allows one class (child class) to **acquire the properties and behaviors (attributes & methods) of another class (parent/base class)**.*
*It helps **reuse code**, **avoid duplication**, and support **extensibility**.*

🔹 ***Why inheritance is useful***

- *You don't need to rewrite the same code again.*
- *You can build new features on top of existing classes.*
- *Helps maintain clean and scalable code.*

✓***Types of inheritance in Python (just names)***

- *Single inheritance*
- *Multiple inheritance*
- *Multilevel inheritance*
- *Hierarchical inheritance*
- *Hybrid inheritance*

🔥 ***Example: Single Inheritance***

```
# Parent class
class Animal:
    def sound(self):
        print("Animals make different sounds")

# Child class inheriting from Animal
class Dog(Animal):
    def sound(self):
        print("Dog barks")

# Creating objects
a = Animal()
d = Dog()

a.sound()   # Output: Animals make different sounds
d.sound()   # Output: Dog barks
```

🧠 ***How it works***

- *Dog(Animal) means **Dog class inherits the Animal class**.*
- *Because of inheritance, Dog can access methods of Animal.*
- *If the child class defines a method with the same name (sound()), it **overrides** the parent class method — this is called **method overriding**.*

🐾 ***Another Example (Without overriding)***

```
class Vehicle:
```

```
    def start(self):
        print("Vehicle started")

class Car(Vehicle):
    pass  # no new methods or attributes added

c = Car()
c.start()   # Output: Vehicle started
```

Even though Car has no method of its own, it can use the start() method of Vehicle.

🌟 Key takeaway

Concept	Meaning
Parent/Base class	The class whose properties are inherited
Child/Derived class	The class that inherits from another
super()	Used to call parent class constructor/methods
Method overriding	Child class redefines a method from parent

🔥 Bonus example with super()

```
class Person:
    def __init__(self, name):
        self.name = name

class Student(Person):
    def __init__(self, name, roll_no):
        super().__init__(name)   # calling parent constructor
        self.roll_no = roll_no

s = Student("Yogesh", 101)
print(s.name)      # Output: Yogesh
print(s.roll_no)   # Output: 101
```

42. Are access specifiers used in python?

*Yes, Python supports **access specifiers**, but **not in the same strict way as languages like Java or C++**. In Python, access specifiers are more of a **naming convention** rather than enforcement — because Python follows the philosophy:*

"We are all adults here."
Meaning programmers are trusted not to misuse class members.

🔐 ***Access Specifiers in Python***

Access Specifier	***Syntax***	***Meaning***
Public	*variable / method()*	*Accessible everywhere*
Protected	*_variable / _method()*	*Intended to be used within class and subclasses*
Private	*__variable / __method()*	*Name-mangled — cannot be accessed directly outside class*

✓1. Public Members

*By default, all class attributes and methods in Python are **public**.*

```
class Person:
    def __init__(self):
        self.name = "Yogesh"  # public


obj = Person()
print(obj.name)  # Accessible
```

✓2. Protected Members (_)

*Prefixing with one underscore suggests that the member is **intended for internal use**, but it can still be accessed outside the class.*

```
class Person:
    def __init__(self):
        self._age = 21  # protected


obj = Person()
```

print(obj._age) # Technically accessible, but not recommended

---

✓3. Private Members (__)

*Prefixing with **two underscores** triggers **name mangling**, which makes the member hard to access from outside the class.*

class Person:

def __init__(self):

self.__salary = 50000 # private

obj = Person()

print(obj.__salary) # ❌ Error: AttributeError

*To access it (not recommended but possible), we use **name mangling**:*

print(obj._Person__salary) # ✓Accessible using name-mangled form

---

🧠 Why Python uses naming conventions (instead of strict enforcement)

Because Python aims to be:

- *Simple*
- *Flexible*
- *Programmer-friendly*
- *Less rigid than compiled OOP languages*

43. Is it possible to call parent class without its instance creation?

Yes, in Python we can call the parent class without creating its instance.
This is usually done using super() inside the child class, which allows us to access parent class methods or constructors without explicitly creating a parent class object.
Alternatively, we can call the parent class method using ParentClass.method(self).

44. How is an empty class created in python?

An empty class does not have any members defined in it. It is created by using the pass keyword (the pass command does nothing in python). We can create objects for this class outside the class.

For example

class EmptyClassDemo: pass obj=EmptyClassDemo() obj.name="Interviewbit" print("Name created= ",obj.name) Output: Name created = Interviewbi

47. What is init method in python?

The __init__ method in Python is a constructor that gets executed automatically when an object is created.
It is used to initialize the object's attributes and define its initial state.
It does not return any value other than None.

48. How will you check if a class is a child of another class?

This is done by using a method called issubclass() provided by python. The method tells us if any class is a child of another class by returning true or false accordingly.

For example:

class InterviewbitEmployee: # init method / constructor def __init__(self, emp_name): self.emp_name = emp_name # introduce method def introduce(self): print('Hello, I am ', self.emp_name) emp = InterviewbitEmployee('Mr Employee') # __init__ method is called here and initi emp.introduce() class Parent(object): pass class Child(Parent): pass # Driver Code print(issubclass(Child, Parent)) Python Interview Questions #True print(issubclass(Parent, Child)) #False

We can check if an object is an instance of a class by making use of isinstance() method.

Pandas Interview Questions

49. What do you know about pandas?

Pandas is a fast, powerful, and open-source Python library used for data analysis and data manipulation.
It provides easy-to-use data structures like Series (1-dimensional) and DataFrame (2-dimensional), which make it very convenient to work with structured data.

Why Pandas is widely used

- *Makes data analysis easier and faster*
- *Removes the need to write complex loops*
- *Works well in the machine learning pipeline*
- *Widely used in data science, analytics, and AI projects*

💡 *Interview Answer (shortest version)*

Pandas is a Python library used for data analysis and data manipulation.
It provides data structures like Series and DataFrame to load, clean, explore, transform, and analyze structured data efficiently.
It supports operations like filtering, grouping, merging, handling missing values, and importing/exporting data from multiple file formats.

50. Define pandas dataframe.

A Pandas DataFrame is a two-dimensional, tabular data structure in Python that consists of rows and columns, similar to an Excel sheet or SQL table.
It is used to store and manipulate structured data efficiently.

🔑 *Key Points (for interview)*

- *Data is organized in rows and columns*
- *Columns can be of different data types (int, float, string, boolean, etc.)*
- *Labeled axes → index (rows) and column names*
- *Supports powerful operations like filtering, grouping, merging, joins, reshaping, and statistical analysis.*

51. How will you combine different pandas dataframes?

We can combine Pandas DataFrames using concat(), merge(), and join().
concat() stacks DataFrames side-by-side or on top of each other, merge() combines DataFrames based on common columns like SQL joins, and join() merges using index.
The choice depends on how the data is to be combined.

52. Can you create a series from the dictionary object in pandas?

*Yes, we can **create a Pandas Series from a dictionary**.*

✅ ***Interview-style Answer***

Yes, Pandas allows creating a Series using a dictionary.
*The dictionary keys become the **index**, and the dictionary values become the **Series data**.*

53. How will you identify and deal with missing values in a dataframe?

We first identify missing values using isnull(), isna(), sum(), or info() in Pandas.
Then we handle them depending on the situation.
If the number of missing values is small, we can remove those rows or columns using dropna().
Otherwise, we fill them (impute) using appropriate values — mean/median for numerical features, mode for categorical features, or forward/backward fill in time-series.
Advanced techniques like KNN Imputer and Iterative Imputer can also be used when more accuracy is required.

54. What do you understand by reindexing in pandas?

Reindexing is the process of modifying the index (rows or columns) of a DataFrame or Series to a new set of labels.
If a label is not present in the old index, Pandas adds it and fills missing values with NaN by default.
Reindexing is helpful for reordering data, aligning datasets, and handling new indexes.

55. How to add new column to pandas dataframe?

In Pandas, we can add a new column to a DataFrame in multiple ways, depending on how the values are created or assigned.

🔹 *1. Adding a column by assigning a list / Series / scalar*

```
df['new_col'] = [10, 20, 30]       # list
df['status'] = "Active"            # scalar → same value for all rows
```

🔹 *2. Adding a column based on existing columns*

```
df['total'] = df['price'] + df['tax']
```

🔹 *3. Using assign()*

```
df = df.assign(discount = df['price'] * 0.10)
```

🔹 *4. Insert at a specific position using insert()*

```
df.insert(1, 'age', [25, 30, 28])    # index 1 → second column
```

🔹 *5. Adding a column using loc*

```
df.loc[:, 'grade'] = ['A', 'B', 'C']
```

📌 *Interview-friendly summary*

We can add a new column to a Pandas DataFrame by assigning values directly (df['col'] = ...), using assign(), insert() to add at a specific position, or by using loc.
The method depends on whether we want to add values, compute from existing columns, or place it at a particular index.

56. How will you delete indices, rows and columns from a dataframe?

We can delete rows and columns using the drop() function.
axis=0 is used for rows and axis=1 for columns.
Columns can also be deleted using del or pop.
For deleting index and regenerating default index we use reset_index(drop=True).

57. Can you get items of series A that are not available in another series B?

To get items of Series A that are not present in Series B:

- *Use A[~A.isin(B)] (most common)*
- *Or A.difference(B) for direct set difference*

58. How will you get the items that are not common to both the given series A and B?

To get items **not common** to both Series A and Series B:

Method	Expression
Using isin	`pd.concat([A[~A.isin(B)], B[~B.isin(A)]])`
Using built-in function	`A.symmetric_difference(B)`

- 📌 The result contains values present in **A or B but not in both**.

59. While importing data from different sources, can the pandas library recognize dates?

✅ *Yes. Pandas can automatically recognize and parse dates while importing data.*

When reading data (CSV, Excel, SQL, JSON, etc.), we can convert columns into datetime format using the parse_dates parameter.

🔹 *Example with read_csv()*

```
df = pd.read_csv("sales.csv", parse_dates=['order_date'])
```

📌 *parse_dates tells Pandas to convert the column into datetime dtype instead of treating it as a string.*

🔹 *If dates exist in multiple columns (day, month, year)*

```
df = pd.read_csv("data.csv", parse_dates=[[ 'day', 'month', 'year' ]])
```

🔹 *Handling custom date formats*

```
df = pd.read_csv("file.csv", parse_dates=['date'], dayfirst=True)
```

🔥 *Why this matters (strong interview point)*

✓*Correctly parsing dates allows proper time-series analysis*
✓*Enables operations like filtering by month/year, resampling, sorting, and rolling window analysis*
✓*Avoids treating dates as plain text*

💡 *One-liner to impress interviewer*

Yes. Pandas can recognize dates during import using parse_dates. It converts date columns into datetime64[ns], enabling efficient time-series operations.

60. What do you understand by NumPy?

✅ ***NumPy (Numerical Python) is a fundamental Python library used for high-performance numerical and scientific computing.***

It provides a powerful ***N-dimensional array object (ndarray)*** *that is* ***much faster and more memory-efficient than Python lists****, especially for mathematical operations.*

🔥 ***Why NumPy is important (high-impact interview points)***

Feature	***Benefit***
ndarray	*Fast vector & matrix operations*
Broadcasting	*Apply operations over arrays without loops*
Universal functions (ufuncs)	*Optimized mathematical functions*
Integration	*Works with Pandas, Scikit-learn, TensorFlow, PyTorch etc.*
Memory efficiency	*Uses contiguous blocks of memory, unlike Python lists*

🧠 ***Example to show performance difference***

```
import numpy as np


arr = np.array([1, 2, 3])

print(arr * 2)     # vectorized operation
```

Output:

```
[2 4 6]
```

📌 *No loops required — NumPy performs* ***vectorized computation****, which is why it is fast.*

💡 ***One-liner to impress interviewer***

NumPy is the core library for numerical computing in Python, offering fast and memory-efficient multidimensional arrays and vectorized mathematical operations — forming the foundation for data science and machine learning.

61. How are NumPy arrays advantageous over python lists?

✅ ***NumPy arrays are much more efficient than Python lists in terms of speed, memory usage, functionality, and mathematical operations.***

🔥 ***Key Advantages (exact points interviewers expect)***

Advantage	***Why it matters***
Faster computation	*NumPy is implemented in C → vectorized operations without Python loops*
Less memory usage	*Stores elements in continuous memory blocks with fixed datatypes*
Supports mathematical & matrix operations	*Built-in support for linear algebra, statistics, Fourier transform, etc.*
Broadcasting	*Apply operations across arrays of different shapes without manual looping*
Convenience	*Hundreds of optimized functions (ufuncs) for scientific computing*
Better integration	*Works seamlessly with Pandas, Scikit-learn, TensorFlow, OpenCV, etc.*

🔥 ***Example to demonstrate speed difference***

```
import numpy as np

arr = np.array([1, 2, 3])

arr = arr * 5   # vectorized → very fast
```

Equivalently in Python list:

```
lst = [1, 2, 3]

lst = [x * 5 for x in lst]   # slow loop
```

📌 *NumPy executes operations in compiled C behind the scenes →* ***huge performance boost****.*

💡 ***One-liner to impress the interviewer***

NumPy arrays outperform Python lists because they support fast, vectorized, memory-efficient numerical operations, making them ideal for data science, machine learning, and scientific **computing.**

63. You are given a numpy array and a new column as inputs. How will you delete the second column and replace the column with a new column value?

```
import numpy as np
#inputs
inputArray = np.array([[35,53,63],[72,12,22],[43,84,56]])
new_col = np.array([[20,30,40]])
# delete 2nd column
arr = np.delete(inputArray , 1, axis = 1)
#insert new_col to array
arr = np.insert(arr , 1, new_col, axis = 1)
print (arr)
```

64. How will you efficiently load data from a text file?

✅ *We can efficiently load data from a text file in NumPy using np.loadtxt() or np.genfromtxt(), depending on whether the file contains missing values.*

🔹 *1. Using np.loadtxt() (Fastest — when data is clean)*

import numpy as np

data = np.loadtxt("data.txt", delimiter=",")

📌 *loadtxt() is very fast and ideal when:*

- *Data is purely numeric*
- *No missing values exist*

🔹 *2. Using np.genfromtxt() (Handles messy/noisy data)*

data = np.genfromtxt("data.txt", delimiter=",", missing_values="NaN", filling_values=0)

📌 *Useful when:*

- *File contains missing/invalid entries*
- *You need to fill missing values automatically*

🔹 *3. For extremely large files (best performance)*

Use memory-mapped loading:

data = np.loadtxt("data.txt", delimiter=",", max_rows=500000)

or

data = np.memmap("data.txt", mode='r')

📌 *memmap allows reading the data without loading whole file into RAM, perfect for large datasets.*

💡 *Interview-winning summary*

Method	*When to use*
np.loadtxt()	*Clean numeric data — fastest*
np.genfromtxt()	*Missing or messy data*
np.memmap()	*Extremely large files that don't fit into RAM*

⚡ *One-liner to impress interviewer*

For efficient text file loading, NumPy provides loadtxt() for clean numeric data and genfromtxt() for data with missing values. For very large files, memmap enables fast loading without consuming full memory.

66. How will you sort the array based on the Nth column?

✅ ***We can sort a 2-D NumPy array based on any column using the argsort() function on that column and then reindex the whole array.***

🔹 ***Example: Sort array by the N-th column***

import numpy as np

arr = np.array([[10, 2, 30],

[40, 1, 20],

[20, 3, 10]])

```
N = 1   # column index to sort by (2nd column)

sorted_arr = arr[arr[:, N].argsort()]

print(sorted_arr)
```

🔹 *Output:*

```
[[40  1 20]

 [10  2 30]

 [20  3 10]]
```

🧠 ***How this works***

Step	***Explanation***
arr[:, N]	*Extract N-th column*
.argsort()	*Returns indices that would sort that column*
arr[...]	*Reorders entire rows based on those sorted indices*

🔥 ***Bonus: Sort in descending order***

```
sorted_arr = arr[arr[:, N].argsort()[::-1]]
```

💡 ***Interview-friendly summary***

To sort a 2-D NumPy array by the N-th column, apply argsort() on that column and reindex the whole array: arr[arr[:, N].argsort()]. Use [::-1] for descending order.

67. How will you find the nearest value in a given numpy array?

We can use the argmin() method of numpy as shown below:

```
import numpy as np
def find_nearest_value(arr, value):
   arr = np.asarray(arr)
   idx = (np.abs(arr - value)).argmin()
   return arr[idx]
#Driver code
arr = np.array([ 0.21169,  0.61391, 0.6341, 0.0131, 0.16541,  0.5645,  0.5742])
value = 0.52
print(find_nearest_value(arr, value)) # Prints 0.5645
```

68. How will you reverse the numpy array using one line of code?

This can be done as shown in the following:

```
reversed_array = arr[::-1]
```

where **arr** = original given array, reverse_array is the resultant after reversing all elements in the input.

69. How will you find the shape of any given NumPy array?

We can use the shape attribute of the numpy array to find the shape. It returns the shape of the array in terms of row count and column count of the array.

```
import numpy as np
arr_two_dim = np.array([("x1","x2", "x3","x4"),
             ("x5","x6", "x7","x8" )])
arr_one_dim = np.array([3,2,4,5,6])
# find and print shape
print("2-D Array Shape: ", arr_two_dim.shape)
print("1-D Array Shape: ", arr_one_dim.shape)
"""
Output:
2-D Array Shape:  (2, 4)
1-D Array Shape:  (5,)
"""
```